UNIVERSIDAD DE ANTIOQUIA
Facultad de Ingeniería

Informática II
Semestre 2026 – 1
Desafío I
Marzo 09, del 2026

Implementación de Tetris mediante manipulación de bits en C++

Este desafío consiste en diseñar e implementar una versión simplificada del juego Tetris, aplicando conceptos avanzados de operadores a nivel de bits (bitwise) y gestión de memoria dinámica, bajo restricciones estrictas de programación estructurada.

Objetivos

- Desarrollar la capacidad de solución de problemas en los estudiantes, enfrentándolos a problemáticas cercanas a situaciones reales.
- Evaluar si el estudiante ha adquirido las destrezas y conocimientos fundamentales de la programación en C++, tales como el uso de estructuras de control (secuenciales, iterativas y condicionales), manejo de tipos de datos, operaciones a nivel de bits, punteros, arreglos, memoria dinámica y funciones.

El objetivo principal de esta actividad es poner a prueba sus habilidades en el análisis de problemas y en el dominio del lenguaje C++. Si ha seguido un proceso disciplinado de aprendizaje a lo largo del semestre, esta es una excelente oportunidad para demostrarlo. Podrá proponer una solución efectiva y obtener un resultado satisfactorio. En caso contrario, esta actividad le permitirá identificar sus debilidades y tomar acciones correctivas con miras a prepararse mejor para enfrentar desafíos similares.

Se recomienda valorar adecuadamente la verdadera complejidad del problema planteado. No se rinda antes de intentarlo ni descarte posibles enfoques sin analizarlos. Descubrirá que, aunque al principio el reto pueda parecer difícil, ya ha desarrollado las herramientas necesarias para enfrentarlo con éxito. Si dedica el tiempo adecuado al análisis del problema, el proceso de codificación será mucho más fluido.

Esperamos que disfrute del desafío propuesto. Le sugerimos leer detenidamente todo el documento antes de comenzar, y asegurarse de entender completamente las instrucciones antes de abordar esta actividad evaluativa.

Fue revisado por los profesores Aníbal Guerra y Augusto Salazar.

Especificaciones

Se solicita desarrollar un motor de juego de Tetris basado en interfaz a través de la consola. A diferencia del juego tradicional, esta versión no incluirá animaciones en tiempo real ni gráficos complejos (manejo únicamente por consola). El flujo se basará en turnos según esta dinámica: el sistema muestra el estado actual del tablero, solicita al usuario una acción y procesa el resultado del movimiento de la pieza actual.

Tablero

- **Dimensiones:** Al iniciar, el programa debe preguntar al usuario el ancho y alto del tablero. El ancho y alto mínimo deben ser de 8 bloques.
- **Validación:** El ancho del tablero debe ser estrictamente un **múltiplo de 8** (para facilitar el manejo de bytes).
- **Lógica basada en operaciones a nivel de Bits:** Debe definir las estructuras de datos que permitan representar de manera eficiente todos los elementos requeridos en la visualización (pantalla, piezas, otros). Esto incluye seleccionar los tipos de datos adecuado para almacenar el contenido mostrado en la pantalla según el ancho elegido, a fin de optimizar el almacenamiento.

Mecánica del Juego

- **Piezas:** Debes definir al menos 5 tipos de piezas según las opciones clásicas. Cada pieza debe ser representada y desplazada utilizando operaciones a nivel de bits (desplazamientos <<, >>, máscaras &, OR |, etc.). Aquí está el detalle de los 7 tetriminos clásicos:
  - **I (Línea o Pieza I):** Recta de cuatro bloques (1x4).
  - **O (Cuadrado):** Bloque de 2x2.
  - **T (Pieza T):** Forma de T. Versátil para encajar en espacios irregulares (3x2).
  - **S (Pieza S):** Forma de S, se orienta hacia la derecha (3x2).
  - **Z (Pieza Z):** Forma de Z, se orienta hacia la izquierda (3x2).
  - **J (Pieza J):** Forma de L invertida o con gancho a la izquierda (2x3).
  - **L (Pieza L):** Forma de L estándar o con gancho a la derecha(2x3).

  Genere las piezas bajo un criterio de aleatoriedad uniforme, y desde la ubicación central en el ancho del tablero. Siga los siguientes patrones de rotación:

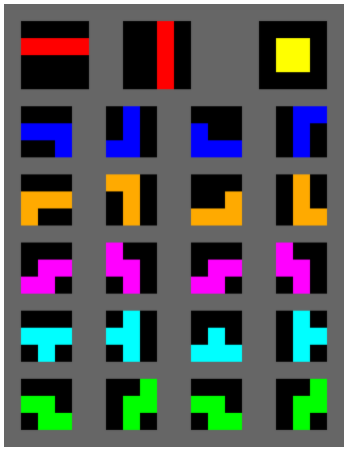

Figura 1. Ejemplo de rotaciones para cada pieza. Tomado de Wikipedia

- **Entrada del Usuario:** En cada turno, el usuario podrá elegir (por teclado):
    - Mover una posición a la izquierda.
    - Mover una posición a la derecha.
    - Bajar una posición.
    - Rotar la pieza (horizontal o verticalmente).

- **Detección de Colisiones:** Se debe verificar si un movimiento es válido comparando los bits de la pieza con los bits ocupados en el tablero. De igual forma considere las fronteras.
- **Limpieza de Filas:** Cuando una fila del tablero tenga todos sus bits en 1, esta debe eliminarse y las superiores deben bajar (usando manipulación de punteros y bits).

Interfaz de usuario

- Cada vez que el tablero cambie, se debe imprimir en consola una representación visual (ejemplo: usar '#' para bloques ocupados y '.' para vacíos). Por ejemplo:

Figura 2. Ejemplo de interfaz de usuario por consola (propia)

- El programa debe indicar claramente cuándo ocurre un *Game Over* (si una pieza nueva no puede colocarse en la parte superior).

Criterios generales de evaluación

1. **Uso de operadores a nivel de bits (30%):** Uso eficiente de máscaras y desplazamientos para colisiones, rotaciones, fijación de piezas, eliminación de filas y detección de *Game Over*.
2. **Gestión eficiente de la memoria (35%):** Implementación y uso correcto de la memoria dinámica. Selección adecuada de las estructuras y tipos de datos. Balancear con la eficiencia en instrucciones.
3. **Lógica de juego (35%):** Funcionamiento correcto de las reglas de Tetris (movimiento, gravedad por turno, eliminación de filas y detección de *Game Over*.

Requisitos mínimos

A continuación, se describen los requisitos que se deben cumplir. El incumplimiento de cualquiera de ellos implica que su nota sea cero.

1. Genere un informe en donde se detalle el desarrollo del proyecto, explique entre otras cosas:
    a. Análisis del problema y consideraciones para la alternativa de solución propuesta.
    b. Esquema donde describa las tareas que usted definió en el desarrollo de los algoritmos.
    c. Algoritmos implementados.
    d. Problemas de desarrollo que afrontó.
    e. Evolución de la solución y consideraciones para tener en cuenta en la implementación.
    f. El informe no debe ser generado por alguna herramienta de IA.
2. La solución debe ser implementada en lenguaje C++ en el *framework* Qt. No incluir sintaxis de ANSI C.
3. La selección del tipo de variables y estructuras de datos debe hacerse responsablemente, pensando en uso eficiente de memoria. Las decisiones tomadas deben estar bien argumentadas.
4. No se pueden usar objetos tipo *string* como parte de la solución.
5. La implementación debe incluir el uso de punteros, arreglos y memoria dinámica.
6. En la implementación no se pueden usar estructuras, librerias no autorizadas, ni STL.
7. Se debe crear un repositorio público en el cual se van a poder cargar todos los archivos relacionados a la solución planteada por usted (informe, código fuente y otros anexos).
8. Una vez cumplida la fecha de entrega no se podrá hacer modificación alguna al repositorio.
9. Se deben hacer *commits* de forma regular (repartidos durante el periodo de tiempo propuesto para las entregas) de tal forma que se evidencie la evolución de la propuesta de solución y su implementación.
10. Se debe adjuntar un enlace de *youtube* a un video que debe incluir lo siguiente:
    a. Presentación de la solución planteada. Análisis realizado y explicación de la arquitectura del sistema (3 minutos máximo).
    b. Demostración de funcionamiento del sistema. Explicar cómo funciona: ejemplos demostrativos (3 minutos máximo).
    c. Explicación del código fuente. Tenga en cuenta que debe justificar la elección de las variables y estructuras de control usados. Por qué eligió uno u otro tipo de variable o estructura de control en cada caso particular y que ventaja ofrecen estos en comparación de otras que también podrían haber sido usados (5 minutos máximo).
    d. La duración total del video no debe exceder 11 minutos ni ser inferior a 5 minutos.
    e. Asegúrese que el video tenga buen sonido y que se puede visualizar con resolución suficiente para apreciar bien los componentes presentados.
    f. El video no debe ser generado por alguna herramienta de IA.
    g. En la elaboración del video deben participar todos los miembros del equipo. La colaboración de cada uno debe ser explícita (a través del audio descriptivo y/o video donde aparezcan).

11. El **plazo de** entrega se divide en dos momentos:
    a. El día **13 de marzo** (11:59 pm), para adjuntar la evidencia del proceso de análisis y diseño de la solución. (Informe preliminar: Contextualización, Análisis, Diseño)
    b. El día **20 de marzo** (11:59 pm), para adjuntar la evidencia del proceso de implementación.

12. En Ude@ se deben adjuntar dos enlaces: uno al repositorio y otro al video, nada más.
13. Para la evaluación del desafío se realizará una sustentación oral en un horario concertado con el profesor. La asistencia a la sustentación es obligatoria. Las sustentaciones que se realicen en modalidad virtual requerirán uso de cámara y el cumplimiento de condiciones de visualización por parte de los estudiantes.

Acotaciones finales

Ante la posibilidad de uso de herramientas de generación automática de soluciones se recuerda al estudiantado la responsabilidad que tiene de demostrar dominio analítico absoluto sobre la solución que entrega, así como control sobre las decisiones tomadas y el proceso de desarrollo ejecutado. Del mismo modo, se espera un conocimiento fluido de las instrucciones y del funcionamiento de su propio código. Todo lo anterior será verificado durante la sustentación, siendo esto un factor fundamental en el proceso de calificación.

La entrega se dará por inválida en cualquiera de los siguientes casos:

- Si se omite la entrega, o se encuentra inaccesible algún enlace a: repositorio, informe, o video.
- Si la solución utiliza instrucciones de ANSI C.
- Si la solución no está debidamente diseñada usando programación modular/multi-archivos (.h, .cpp).
- Si la solución no hace uso de memoria dinámica en el core de la misma.
- Si la solución no hace uso de operadores a nivel de bits en el core de la misma.
- Si el core de la solución se implementa delegando tareas a código de autoría externa.
- Si se incumple cualquier otra restricción establecida en este documento.